TASK 1: CREATE AWS VPC USING TERRAFORM

1. Objective

To create an Amazon Virtual Private Cloud (VPC) using Terraform Infrastructure as Code (IaC).

The VPC provides an isolated virtual networking environment in AWS where resources such as EC2 instances, subnets, route tables, and security groups can be deployed.

2. Purpose

The purpose of this task is to establish the primary network infrastructure required for the Terraform final project.

We use Terraform to automate VPC creation instead of manually creating it through the AWS Management Console.

The VPC will be used in subsequent tasks to create:

- Internet Gateway
- Custom Route Table
- Public Subnet
- Security Group
- Network Interface
- Elastic IP
- Ubuntu EC2 Instance

3. Project Configuration

- Cloud Provider: AWS
- Region: us-east-1 (N. Virginia)
- VPC Name: terraform-final-vpc
- VPC CIDR: 10.0.0.0/16
- DNS Support: Enabled
- DNS Hostnames: Enabled

4. Project Directory Creation

Opened VS Code and selected the Git Bash terminal.

Created the project directory using: mkdir terraform-final-task

Entered the project directory: cd terraform-final-task

Opened the directory in VS Code:

Created the Terraform configuration file: main.tf

Project structure:

terraform-final-task/

└── main.tf

5. Terraform Configuration

File name: **main.tf**

Complete Terraform code:

```
terraform {
  required_providers {
    aws = {
      source  = "hashicorp/aws"
      version = ">= 5.0"
    }
  }
}

provider "aws" {
  region = "us-east-1"
}

# Create VPC
resource "aws_vpc" "my_vpc" {
  cidr_block           = "10.0.0.0/16"
  enable_dns_support   = true
  enable_dns_hostnames = true

  tags = {
    Name = "terraform-final-vpc"
  }
}
```

6. Explanation of Terraform Code

Terraform Block

```
terraform {
  required_providers {
    aws = {
      source  = "hashicorp/aws"
      version = ">= 5.0"
    }
  }
}
```

Purpose:

- Defines Terraform configuration requirements.
- Specifies the AWS provider.
- Downloads the required provider plugin.

AWS Provider Block

```
provider "aws" {
  region = "us-east-1"
}
```

Purpose:

- Connects Terraform to AWS.
- Specifies N. Virginia as the deployment region.

VPC Resource Block

```
resource "aws_vpc" "my_vpc" {
  cidr_block         = "10.0.0.0/16"
  enable_dns_support   = true
  enable_dns_hostnames = true

  tags = {
    Name = "terraform-final-vpc"
  }
}
```

Purpose:

- Creates an AWS VPC.
- Assigns the IPv4 CIDR range 10.0.0.0/16.
- Enables DNS resolution.
- Enables DNS hostnames.
- Assigns a Name tag for identification.

7. Terraform Execution Steps

All commands were executed inside the VS Code Git Bash terminal, from the terraform-final-task directory.

Step 1: Initialize Terraform: terraform init

Step 2: Validate Configuration: terraform validate

Step 3: Generate Execution Plan: terraform plan

Step 4: Create VPC: terraform apply

8. Verification

Verify Using Terraform State: terraform state list

Expected output: aws_vpc.my_vpc

This confirms that Terraform is tracking the created VPC.

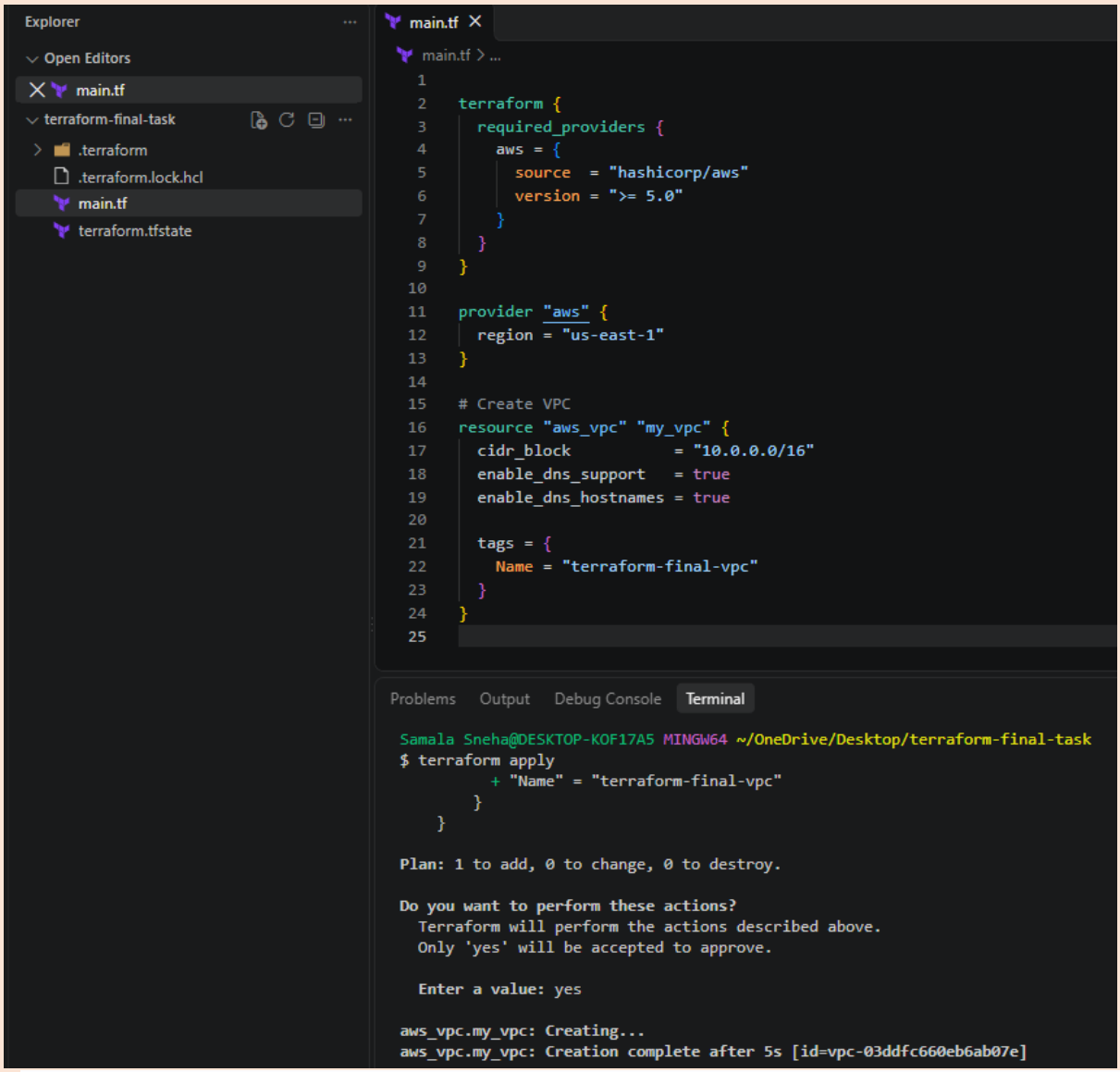
Verify Using AWS Management Console

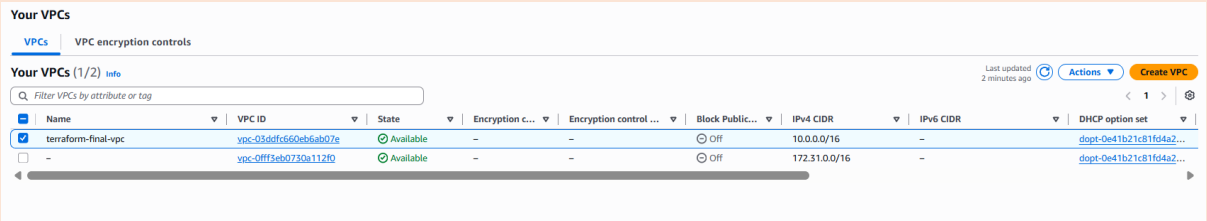
1. Open AWS Management Console.
2. Navigate to the VPC service.
3. Select Your VPCs.
4. Search for terraform-final-vpc.

5. Verify the following:

- VPC state: Available
- IPv4 CIDR: 10.0.0.0/16
- DNS resolution: Enabled
- DNS hostnames: Enabled

9. Result





10. Conclusion

In this task, we successfully created an AWS Virtual Private Cloud using Terraform.

We learned how to configure the AWS provider, define a VPC resource, initialize Terraform, validate configurations, generate execution plans, apply infrastructure changes, and verify deployed resources.

The VPC is now ready to serve as the foundation for the remaining AWS infrastructure.

TASK 2: CREATE INTERNET GATEWAY USING TERRAFORM

1. Objective

To create an AWS Internet Gateway using Terraform and attach it to the VPC created in Task 1.

2. Purpose

An Internet Gateway (IGW) is an AWS networking component that enables communication between resources inside a VPC and the internet.

The VPC created in Task 1 provides an isolated virtual network. To enable internet connectivity for resources inside this VPC, we need an Internet Gateway.

In this task, we use Terraform to create an Internet Gateway and attach it to our existing VPC.

The Internet Gateway will be used in subsequent tasks to provide internet connectivity to the public subnet and Ubuntu EC2 instance.

Note: An Internet Gateway alone does not provide internet access. A suitable route table, subnet association, public IP addressing, and security group rules are also required.

3. Terraform Configuration

Added the following resource block below the existing VPC resource.

```
# Create Internet Gateway
resource "aws_internet_gateway" "my_igw" {
 vpc_id = aws_vpc.my_vpc.id

 tags = {
  Name = "terraform-final-igw"
 }
}
```

4. Complete Terraform Configuration

File name: **main.tf**

```
terraform {
 required_providers {
  aws = {
   source  = "hashicorp/aws"
   version = ">= 5.0"
  }
 }
}

provider "aws" {
```

```
  region = "us-east-1"
}

# Create VPC
resource "aws_vpc" "my_vpc" {
  cidr_block           = "10.0.0.0/16"
  enable_dns_support   = true
  enable_dns_hostnames = true

  tags = {
    Name = "terraform-final-vpc"
  }
}

# Create Internet Gateway
resource "aws_internet_gateway" "my_igw" {
  vpc_id = aws_vpc.my_vpc.id

  tags = {
    Name = "terraform-final-igw"
  }
}
```

5. Explanation of Terraform Configuration

Resource Declaration

```
resource "aws_internet_gateway" "my_igw"
```

- aws_internet_gateway: AWS resource type.
- my_igw: Terraform local resource name.

VPC Attachment

```
vpc_id = aws_vpc.my_vpc.id
```

This line attaches the Internet Gateway to the VPC created in Task 1.

Explanation:

- aws_vpc: Refers to the VPC resource type.
- my_vpc: Refers to the Terraform resource name.
- id: Retrieves the actual VPC ID generated by AWS.

Terraform automatically identifies this resource dependency and creates the VPC before creating the Internet Gateway.

Tags

```
tags = {
  Name = "terraform-final-igw"
}
```

Purpose: Assigns a recognizable name to the Internet Gateway in AWS.

6. Terraform Execution Steps

All commands were executed from the existing terraform-final-task directory using the VS Code Git Bash terminal.

Step 1: Format Configuration: terraform fmt

Step 2: Validate Configuration: terraform validate

Step 3: Generate Execution Plan: terraform plan

Expected result: Plan: 1 to add, 0 to change, 0 to destroy.

Step 4: Create Internet Gateway: terraform apply

7. Verification

Method 1: Terraform State Verification: terraform state list

Expected output:

- aws_internet_gateway.my_igw
- aws_vpc.my_vpc

This confirms that Terraform is tracking both the VPC and Internet Gateway.

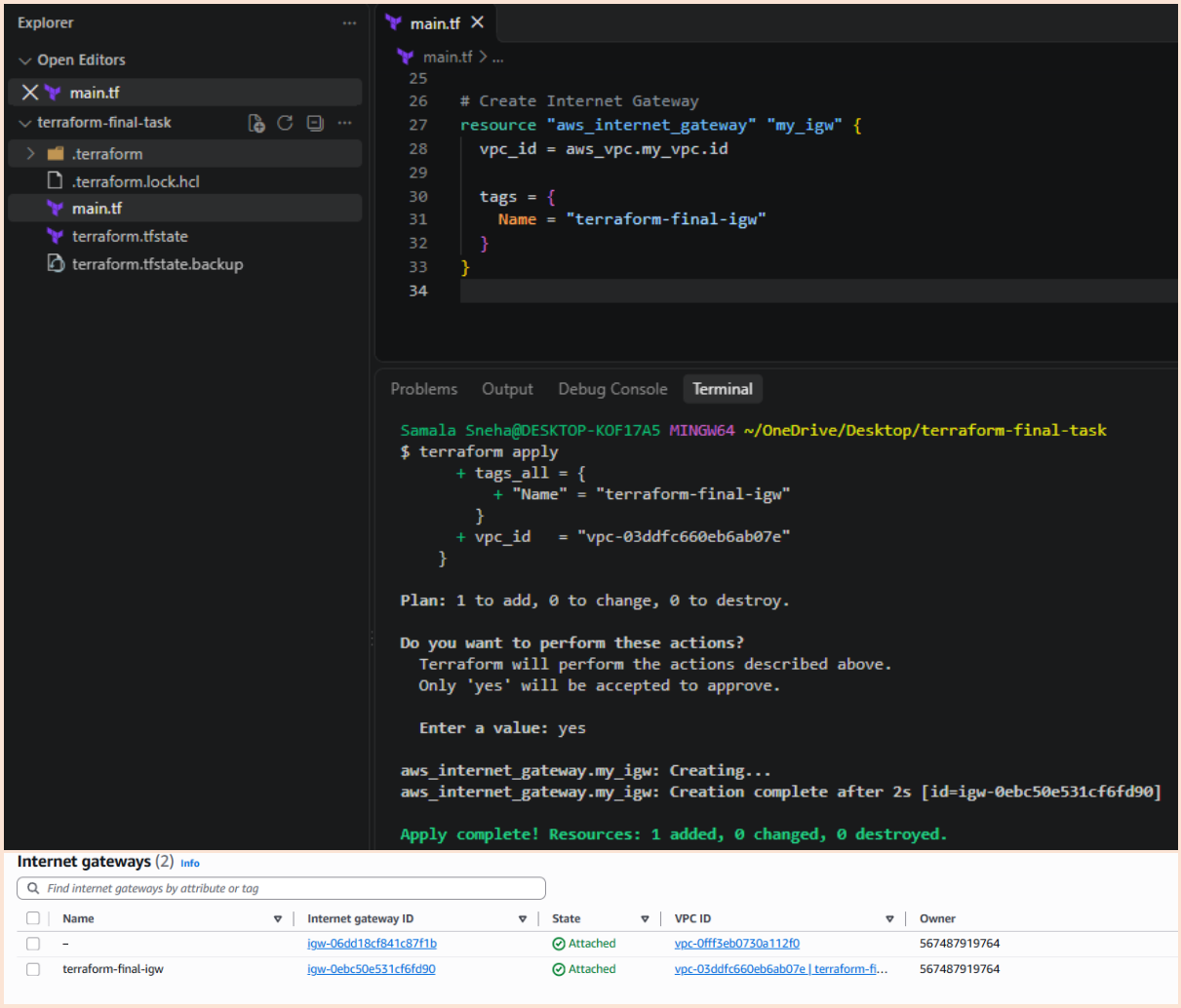
Method 2: AWS Management Console Verification

1. Logged in to AWS Management Console.
2. Opened the VPC service.
3. Selected Internet Gateways.
4. Located terraform-final-igw.
5. Verified that the Internet Gateway was attached to terraform-final-vpc.

Expected status:

- Name: terraform-final-igw
- State: Attached
- VPC: terraform-final-vpc

8. Result



9. Conclusion

In this task, we successfully created and attached an AWS Internet Gateway to the VPC using Terraform.

We learned how to define an Internet Gateway resource, reference an existing Terraform resource using its resource ID, establish resource dependencies, and deploy additional infrastructure without recreating existing resources.

The Internet Gateway is now ready to be used by the custom route table in the next task.

TASK 3: CREATE CUSTOM ROUTE TABLE USING TERRAFORM

1. Objective

To create a custom route table inside an existing AWS VPC using Terraform and configure a default internet route through the Internet Gateway.

2. Purpose

A route table is a collection of routing rules that determines where network traffic from a subnet is directed.

AWS automatically creates a main route table when a VPC is created. However, in this task, we create a separate custom route table to control the traffic of our future public subnet.

We configure the following routes:

1. Local Route: 10.0.0.0/16 → local
2. Internet Route: 0.0.0.0/0 → Internet Gateway

The local route allows communication between resources within the VPC.

The internet route directs internet-bound IPv4 traffic through the Internet Gateway.

3. Terraform Configuration

A. Create Custom Route Table

```
# Create Custom Route Table
resource "aws_route_table" "my_route_table" {
  vpc_id = aws_vpc.my_vpc.id

  tags = {
    Name = "terraform-final-rt"
  }
}
```

B. Create Internet Route

```
# Create Internet Route
resource "aws_route" "internet_route" {
  route_table_id         = aws_route_table.my_route_table.id
  destination_cidr_block = "0.0.0.0/0"
  gateway_id             = aws_internet_gateway.my_igw.id
}
```

4. Explanation of Terraform Code

Custom Route Table Resource

```
resource "aws_route_table" "my_route_table"
```

- aws_route_table: AWS Terraform resource type.
- my_route_table: Terraform local resource name.

VPC Reference

vpc_id = aws_vpc.my_vpc.id

Attaches the custom route table to the VPC created in Task 1.

Internet Route Resource

resource "aws_route" "internet_route"

Creates an individual routing rule in the custom route table.

Route Table ID

route_table_id = aws_route_table.my_route_table.id

Identifies the custom route table in which the route must be created.

Destination CIDR

destination_cidr_block = "0.0.0.0/0"

Represents all IPv4 destinations.

Internet Gateway Reference

gateway_id = aws_internet_gateway.my_igw.id

Specifies the Internet Gateway created in Task 2 as the target for internet-bound traffic.

Terraform automatically identifies the resource dependencies.

5. Terraform Execution Steps

Step 1: Format Terraform Configuration: terraform fmt

Step 2: Validate Terraform Configuration: terraform validate

Step 3: Generate Execution Plan: terraform plan
Plan: 2 to add, 0 to change, 0 to destroy.

Step 4: Apply Terraform Configuration: terraform apply

6. Verification
Terraform State: terraform state list

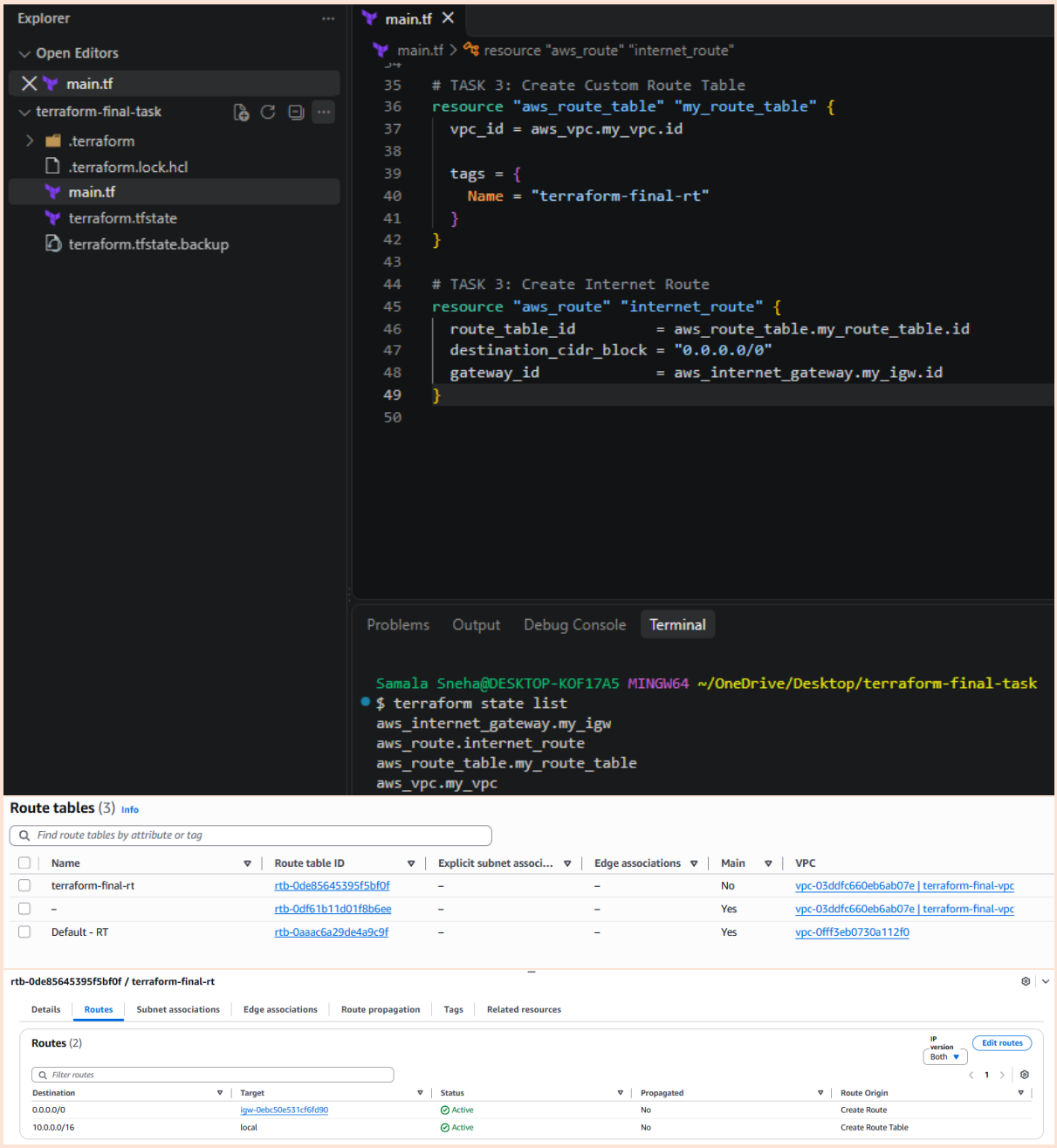
Expected resources:

- aws_internet_gateway.my_igw
- aws_route.internet_route
- aws_route_table.my_route_table
- aws_vpc.my_vpc

Purpose:

Confirms that Terraform tracks all four infrastructure resources.

7. Result



8. Conclusion

In this task, we successfully created a custom route table and configured an internet route using Terraform.

We learned how to create AWS routing resources, reference existing Terraform resources, configure destination CIDR blocks, and direct internet-bound traffic through an Internet Gateway.

The custom route table is now ready to be associated with a public subnet in the next task.

TASK 4: CREATE PUBLIC SUBNET USING TERRAFORM

1. Objective

To create a public subnet inside the existing AWS VPC using Terraform.

The subnet is configured with a CIDR block of 10.0.1.0/24 and automatic public IPv4 address assignment.

2. Purpose

A subnet is a logical subdivision of a VPC's IP address range.

It allows us to organize AWS resources into smaller network segments.

In this task, we created a public subnet inside our existing VPC.

The subnet will be used to launch the Ubuntu EC2 instance in the final task.

We enabled automatic public IP assignment so that EC2 instances launched in this subnet can receive public IPv4 addresses by default.

Note: A subnet is considered public when its associated route table has a route to an Internet Gateway. The association is completed in Task 5.

3. Terraform Configuration

Added the following resource block:

```
# Create Public Subnet
resource "aws_subnet" "my_subnet" {
  vpc_id                  = aws_vpc.my_vpc.id
  cidr_block              = "10.0.1.0/24"
  availability_zone       = "us-east-1a"
  map_public_ip_on_launch = true

  tags = {
    Name = "terraform-final-subnet"
  }
}
```

4. Explanation of Terraform Code

Resource Declaration

```
resource "aws_subnet" "my_subnet"
```

- aws_subnet: AWS Terraform resource type.
- my_subnet: Terraform local resource name.

VPC Reference

```
vpc_id = aws_vpc.my_vpc.id
```

Specifies the existing VPC in which the subnet must be created.

CIDR Block

```
cidr_block = "10.0.1.0/24"
```

AWS reserves five IPv4 addresses in every subnet, leaving 251 available addresses.

Availability Zone

```
availability_zone = "us-east-1a"
```

Specifies the Availability Zone in which AWS creates the subnet.

Automatic Public IPv4 Assignment

```
map_public_ip_on_launch = true
```

Automatically assigns public IPv4 addresses to instances launched into this subnet, unless overridden by network interface settings.

Tags

```
tags = {
  Name = "terraform-final-subnet"
}
```

Assigns a recognizable name to the subnet.

5. Terraform Execution Steps

Step 1: Format Terraform Configuration: terraform fmt

Step 2: Validate Terraform Configuration: terraform validate

Step 3: Generate Execution Plan: terraform plan

Expected result: Plan: 1 to add, 0 to change, 0 to destroy.

Step 4: Create Subnet: terraform apply

6. Verification

Terraform State: terraform state list

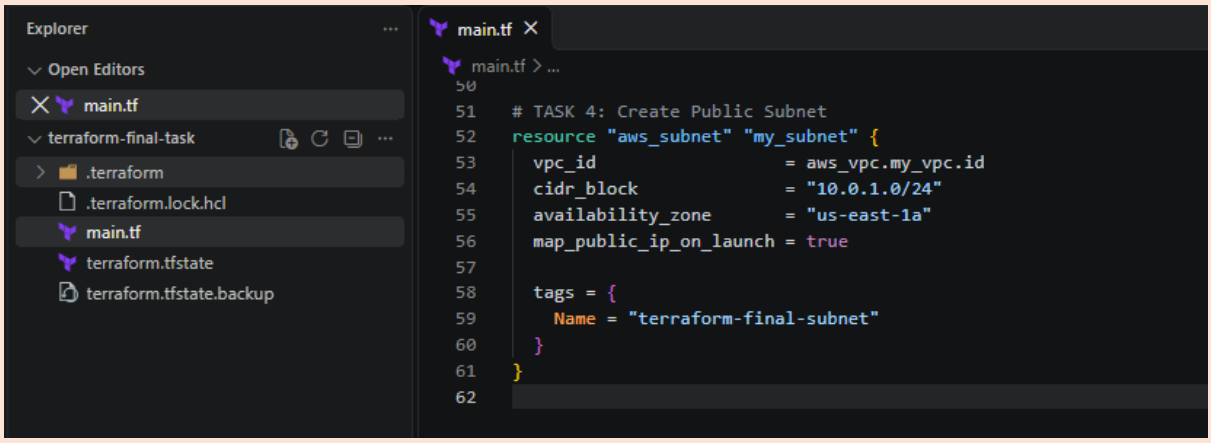
Expected resources:

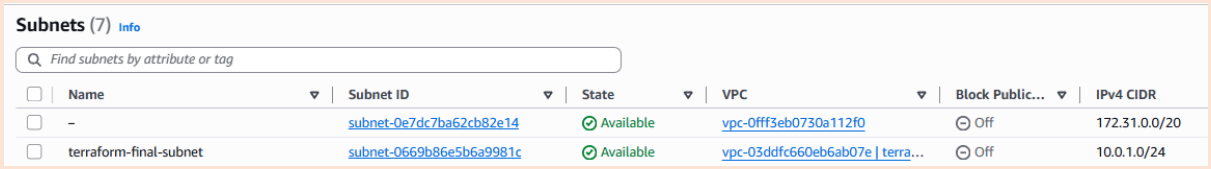
- aws_internet_gateway.my_igw
- aws_route.internet_route
- aws_route_table.my_route_table
- aws_subnet.my_subnet
- aws_vpc.my_vpc

Purpose:

Confirms that Terraform tracks all five infrastructure resources.

7. Result





8. Conclusion

In this task, we successfully created a subnet inside an existing AWS VPC using Terraform.

We learned how to define subnet resources, assign CIDR blocks, select Availability Zones, and enable automatic public IPv4 address assignment.

The subnet is now ready to be associated with the custom route table in the next task.

Task 5: Associate Subnet with Custom Route Table Using Terraform

1. Objective

To associate the subnet terraform-final-subnet with the custom route table terraform-final-rt using Terraform.

2. Purpose

A subnet must be associated with a route table to determine how network traffic is routed.

In the previous tasks, we created:

- A VPC.
- An Internet Gateway.
- A custom route table.
- A public subnet.
- A route in the custom route table directing internet-bound traffic (0.0.0.0/0) to the Internet Gateway.

In this task, we associate the subnet with the custom route table.

This allows resources launched in the subnet to use the routes defined in that route table.

3. Implementation

Step 1: Open the Terraform project

Open the existing terraform-final-task project in VS Code. Open the existing **main.tf** file.

Step 2: Add the subnet association resource

```
# TASK 5: Associate Subnet with Route Table
resource "aws_route_table_association" "subnet_association" {
  subnet_id      = aws_subnet.my_subnet.id
  route_table_id = aws_route_table.my_route_table.id
}
```

Step 3: Understand the configuration

- aws_route_table_association is the Terraform resource used to associate a subnet with a route table.
- subnet_id references the subnet created in Task 4.
- route_table_id references the custom route table created in Task 3.
- Terraform references are used instead of manually entering AWS resource IDs.

Step 4: Format the Terraform configuration: terraform fmt

Step 5: Validate the configuration: terraform validate

Step 6: Review the execution plan: terraform plan

Expected plan summary: Plan: 1 to add, 0 to change, 0 to destroy.

Step 7: Apply the configuration: terraform apply

4. Verification

Method 1: Verify using Terraform state: terraform state list

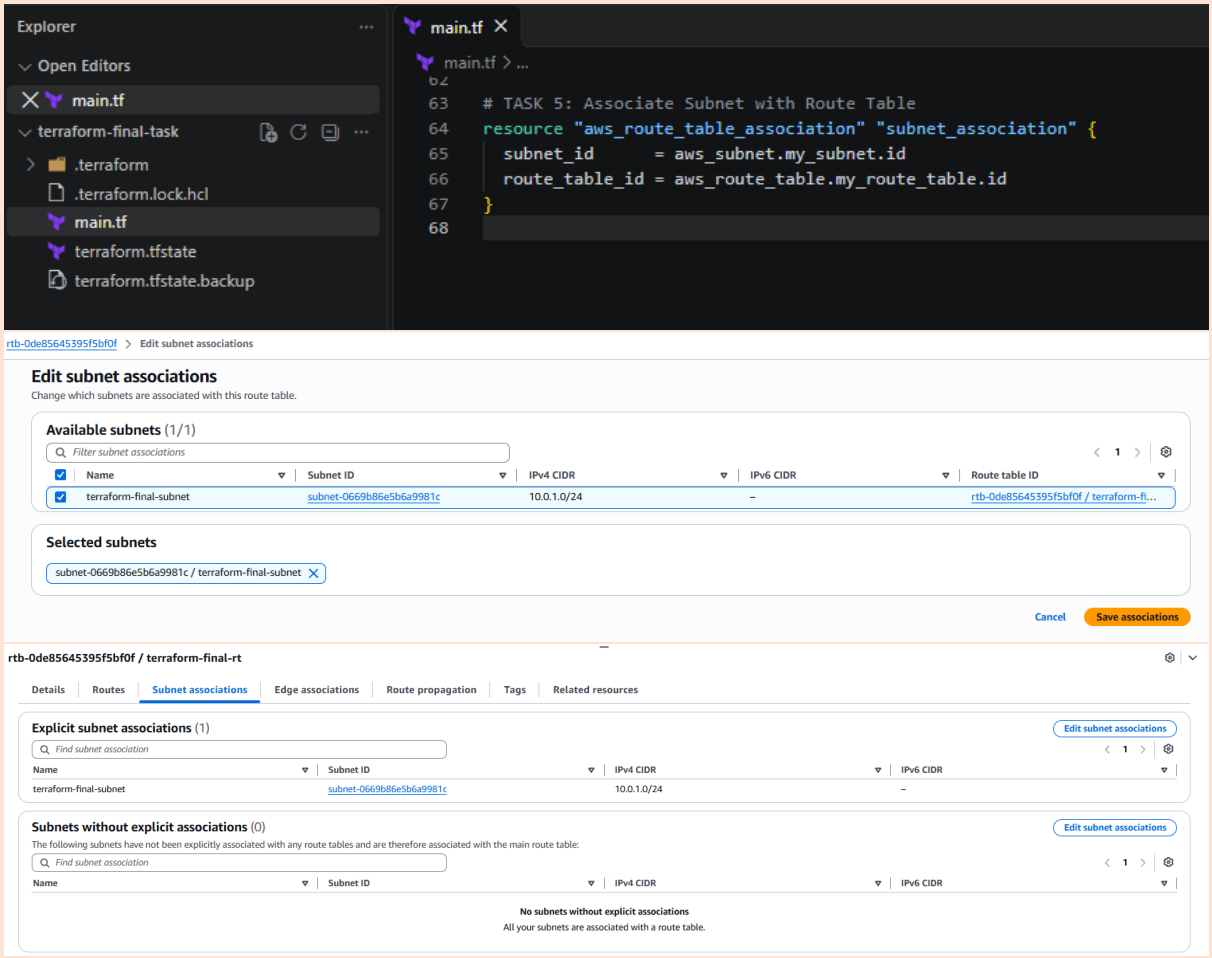
The state should include:

- aws_internet_gateway.my_igw
- aws_route.internet_route
- aws_route_table.my_route_table
- aws_route_table_association.subnet_association
- aws_subnet.my_subnet
- aws_vpc.my_vpc

Method 2: Verify using AWS Management Console

1. Sign in to the AWS Management Console.
2. Open the VPC service.
3. Select **Route Tables** from the left navigation.
4. Select terraform-final-rt.
5. Open the **Subnet associations** tab.
6. Under explicit subnet associations, verify that terraform-final-subnet is listed.

5. Result



6. Conclusion

In this task, we used the Terraform aws_route_table_association resource to associate an existing subnet with an existing custom route table.

This completed the subnet and route-table association required for the public network configuration.

The infrastructure created in Tasks 1–5 is retained in the existing Terraform project and state.

Task 6: Create Security Group Allowing SSH, HTTP and HTTPS

1. Objective

To create an AWS Security Group inside the existing VPC using Terraform and configure inbound rules to allow SSH, HTTP and HTTPS traffic.

2. Purpose

A Security Group acts as a virtual firewall for AWS resources such as EC2 instances and Network Interfaces.

It controls incoming and outgoing network traffic.

In this task, we created a Security Group with the following inbound rules:

Port	Protocol	Purpose	Source
22	TCP	SSH access	124.123.159.170/32
80	TCP	HTTP access	0.0.0.0/0
443	TCP	HTTPS access	0.0.0.0/0

SSH access is restricted to the user's public IP address for improved security.

Outbound traffic is allowed to all IPv4 addresses so that the Ubuntu server can download packages and updates.

3. Implementation

Step 1: Identify the public IP address

Executed the following command in the VS Code Git Bash terminal:
curl -4 https://checkip.amazonaws.com

Output: you get different output{124.123.159.170}

This public IP address was used to restrict SSH access.

Step 2: Configure the Security Group

Opened the existing main.tf file and added the following resource:

```
# TASK 6: Create Security Group

resource "aws_security_group" "my_sg" {
  name        = "terraform-final-sg"
  description = "Allow SSH, HTTP and HTTPS traffic"
  vpc_id      = aws_vpc.my_vpc.id

  # Allow SSH only from my public IP
  ingress {
    description = "SSH from my public IP"
    from_port   = 22
    to_port     = 22
    protocol    = "tcp"
    cidr_blocks = ["124.123.159.170/32"]
  }

  # Allow HTTP from anywhere
  ingress {
    description = "HTTP access"
    from_port   = 80
    to_port     = 80
    protocol    = "tcp"
    cidr_blocks = ["0.0.0.0/0"]
  }

  # Allow HTTPS from anywhere
```

```
  ingress {
    description = "HTTPS access"
    from_port   = 443
    to_port     = 443
    protocol    = "tcp"
    cidr_blocks = ["0.0.0.0/0"]
  }

  # Allow all outbound traffic
  egress {
    from_port   = 0
    to_port     = 0
    protocol    = "-1"
    cidr_blocks = ["0.0.0.0/0"]
  }

  tags = {
    Name = "terraform-final-sg"
  }
}
```

Step 3: Understand the configuration

- aws_security_group: Terraform resource used to create an AWS Security Group.
- name: Specifies the Security Group name.
- description: Describes the purpose of the Security Group.
- vpc_id: Associates the Security Group with the previously created VPC.
- ingress: Defines incoming traffic rules.
- from_port: Specifies the starting port.
- to_port: Specifies the ending port.
- protocol: Defines the network protocol.
- cidr_blocks: Defines the allowed IPv4 source addresses.
- egress: Defines outgoing traffic rules.
- tags: Assigns a recognizable name to the Security Group.

Step 4: Format the configuration: terraform fmt

Step 5: Validate the configuration: terraform validate

Step 6: Review the Terraform execution plan: terraform plan

Expected summary: Plan: 1 to add, 0 to change, 0 to destroy.

This confirmed that Terraform planned to create only the Security Group.

Step 7: Apply the configuration: terraform apply

4. Verification

Method 1: Terraform state: terraform state list

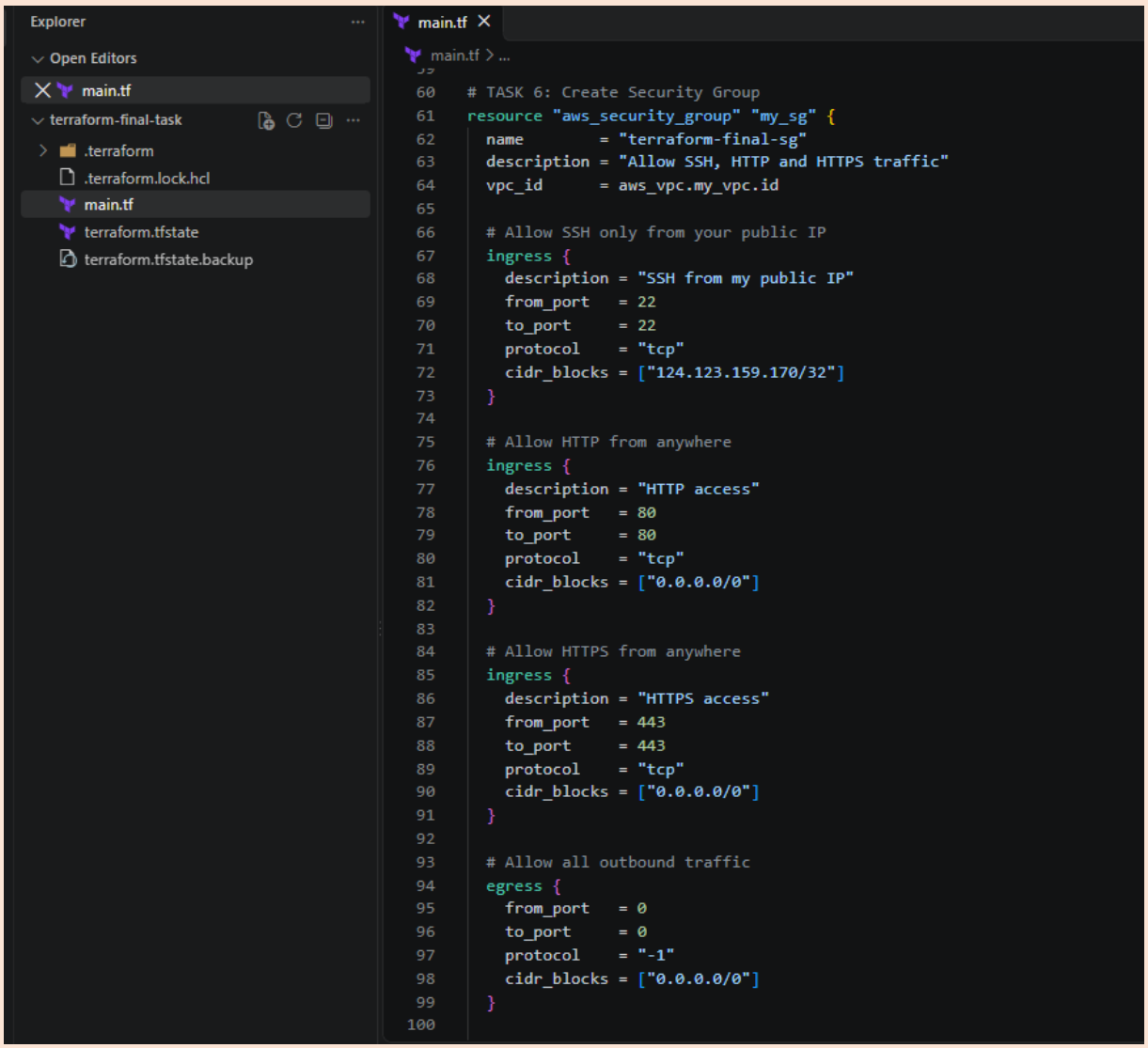
Verified that the following resource was present: aws_security_group.my_sg

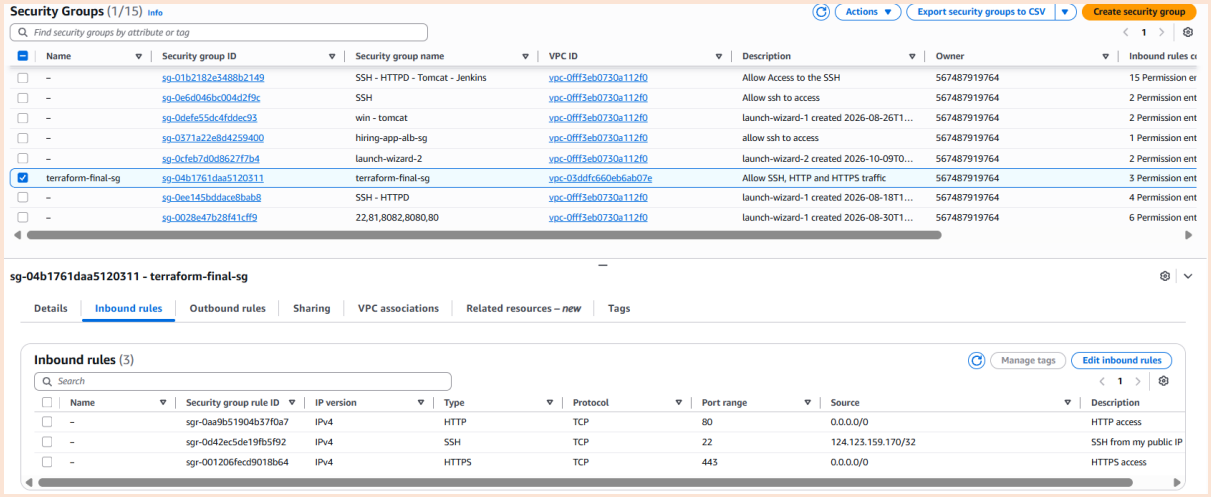
The previously created VPC, Internet Gateway, route table, route, subnet and subnet association remained in the Terraform state.

Method 2: AWS Management Console

1. Opened the AWS Management Console.
2. Navigated to the EC2 service.
3. Selected Security Groups under Network & Security.
4. Located terraform-final-sg.
5. Opened the Security Group.
6. Checked the Inbound rules tab.
7. Verified ports 22, 80 and 443.
8. Checked the Outbound rules tab.

5. Result





6. Conclusion

In this task, we successfully created an AWS Security Group using Terraform.

We configured inbound rules for SSH, HTTP and HTTPS and configured outbound access for the server.

We also restricted SSH access to a specific public IP address instead of exposing port 22 to the entire internet.

The Security Group is now available for attachment to the Network Interface and EC2 instance in the upcoming tasks.

Task 7: Create Network Interface with a Private IP Using Terraform

1. Objective

To create an AWS Elastic Network Interface (ENI) inside the existing subnet and assign a specific private IPv4 address using Terraform.

2. Purpose

An Elastic Network Interface (ENI) is a virtual network interface attached to an EC2 instance.

It provides network connectivity and contains network attributes such as:

- Private IPv4 address.
- Security Group association.
- Subnet association.
- MAC address.
- Network Interface ID.

In this task, we created a Network Interface inside the subnet terraform-final-subnet and assigned the private IP address 10.0.1.50.

We also attached the Security Group created in Task 6.

3. Prerequisites

The following resources must already exist:

Resource	Name
VPC	terraform-final-vpc
Subnet	terraform-final-subnet
Security Group	terraform-final-sg
Route Table	terraform-final-rt
Route Table Association	subnet_association

Additional configuration:

- AWS Region: us-east-1
- Subnet CIDR: 10.0.1.0/24
- Assigned private IP: 10.0.1.50

What is ENI in this Topic?

When you created your subnet with the CIDR block 10.0.1.0/24, AWS allocated a block of 256 total IP addresses (from 10.0.1.0 to 10.0.1.255).

When you manually specified private_ips = ["10.0.1.50"] in your Network Interface configuration, you were picking one specific IP address directly out of that pool of 256 available IPs.

A quick important note on those 256 IPs:

Even though a /24 subnet mathematically contains 256 addresses, AWS actually reserves 5 of those IPs for internal network housekeeping, meaning you have 251 usable IPs for your instances and network cards:

• 10.0.1.0 – Network address (reserved by AWS).

• 10.0.1.1 – VPC router / default gateway (reserved by AWS).

• 10.0.1.2 – Reserved for AWS DNS server mapping.

• 10.0.1.3 – Reserved for future AWS use.

• 10.0.1.255 – Network broadcast address (reserved by AWS).

So your custom IP 10.0.1.50 sits safely inside the usable pool (10.0.1.4 to 10.0.1.254), and AWS marks it as "taken" so no other server in that subnet can accidentally use it.

4. Implementation

Step 1: Open the existing Terraform project

Open the existing terraform-final-task project in VS Code. Open the **main.tf** file.

Do not create a new project or remove the existing resource configurations.

Step 2: Add the Network Interface resource

Added the following configuration at the end of main.tf:

```
# TASK 7: Create Network Interface

resource "aws_network_interface" "my_nic" {
  subnet_id       = aws_subnet.my_subnet.id
  private_ips     = ["10.0.1.50"]
  security_groups = [aws_security_group.my_sg.id]

  tags = {
    Name = "terraform-final-nic"
  }
}
```

Step 3: Understand the configuration

- aws_network_interface: Terraform resource used to create an Elastic Network Interface.
- subnet_id: References the subnet created in Task 4.
- private_ips: Assigns the private IPv4 address 10.0.1.50.
- security_groups: Attaches the Security Group created in Task 6.
- tags: Assigns a recognizable name to the Network Interface.

Terraform automatically determines the creation order from the resource references.

Step 4: Format the configuration: terraform fmt

Step 5: Validate the configuration: terraform validate

Step 6: Review the Terraform execution plan: terraform plan

Expected plan summary:Plan: 1 to add, 0 to change, 0 to destroy.

Step 7: Apply the configuration: terraform apply

5. Verification

Method 1: Verify using Terraform state

Executed:

terraform state list

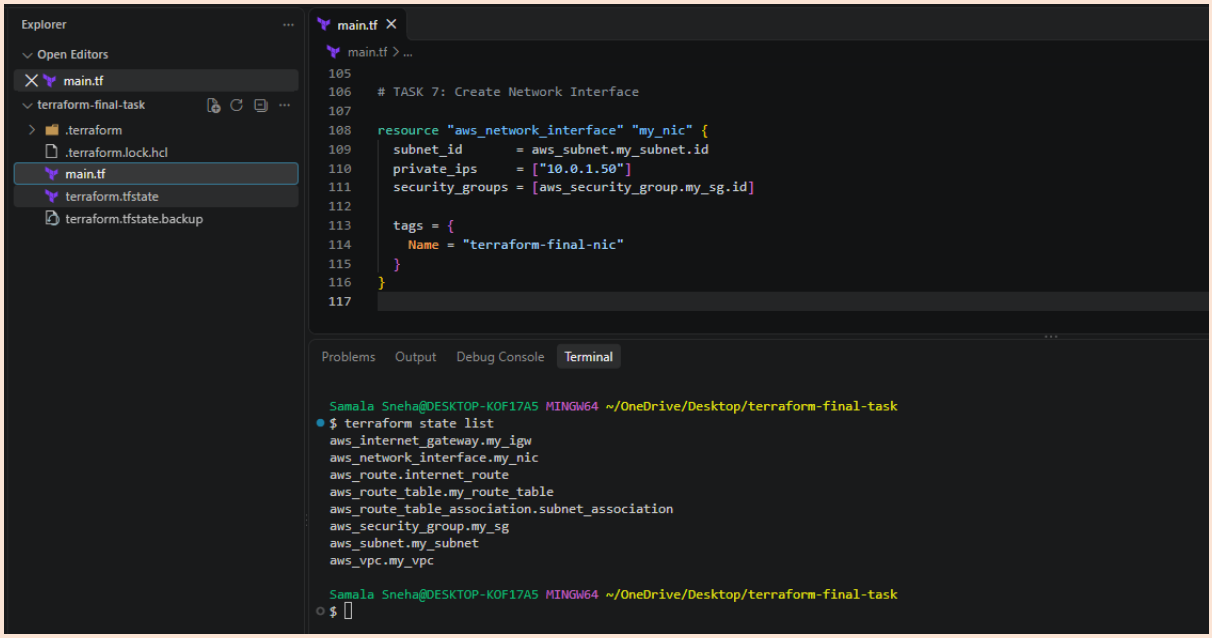
Verified that the following resource was present:

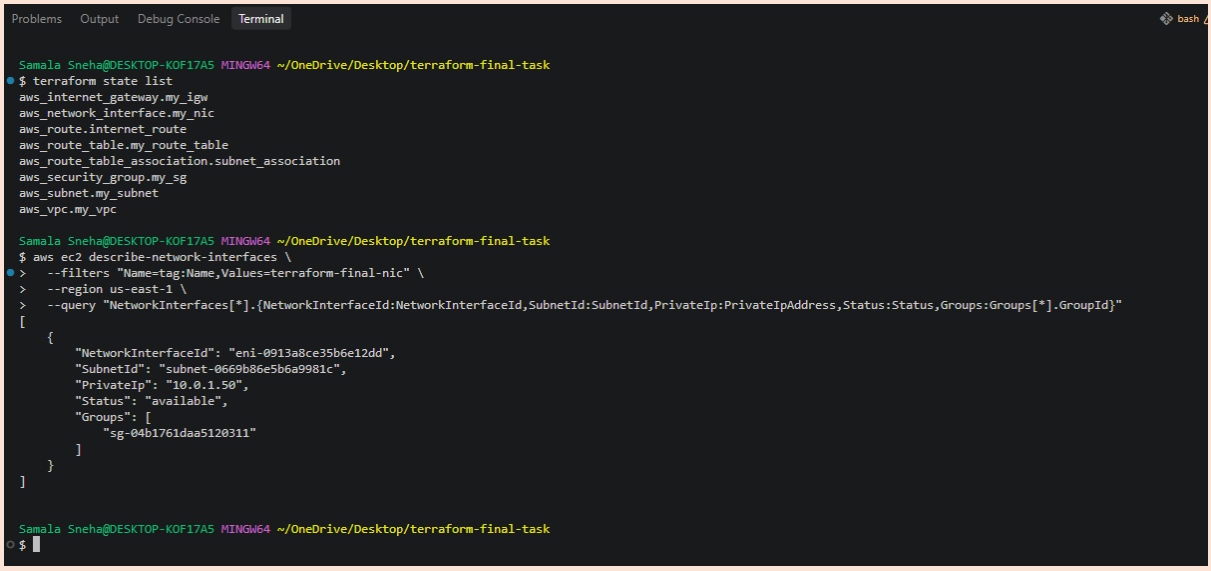
aws_network_interface.my_nic

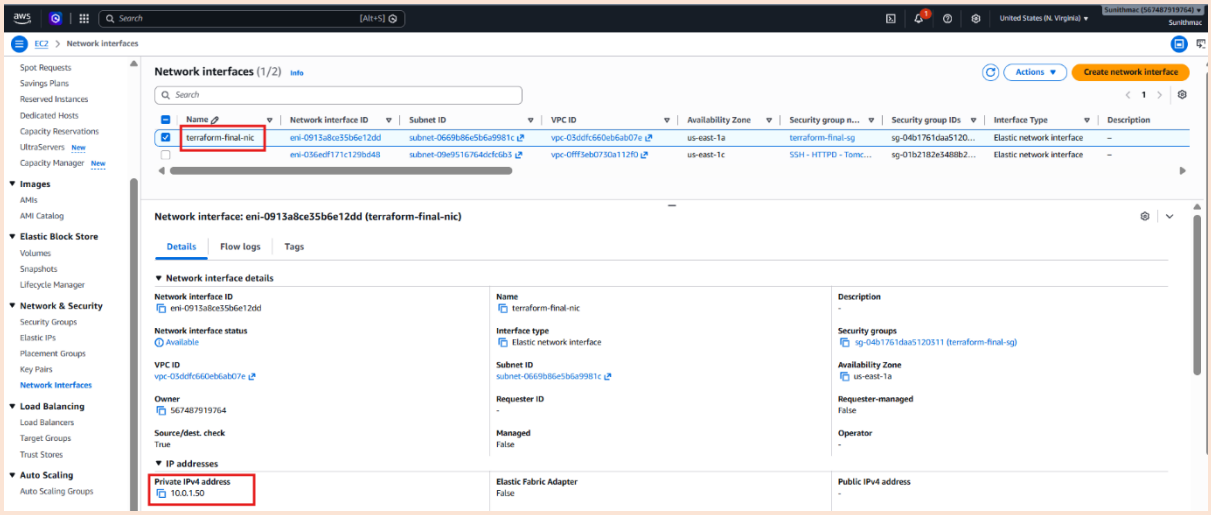
Method 2: Verify using AWS Management Console

1. Open the AWS Management Console.
2. Navigate to the EC2 service.
3. Select **Network Interfaces** under Network & Security.
4. Find terraform-final-nic.
5. Select the Network Interface.
6. Verify its subnet ID.
7. Verify the private IPv4 address.
8. Verify its associated Security Group.

6. Result







7. Conclusion

In this task, we successfully created an Elastic Network Interface using Terraform.

We assigned a private IPv4 address from the existing subnet and associated the Security Group created in Task 6.

The Network Interface is now ready for the Elastic IP assignment in the next task.

Task 8: Allocate and Associate Elastic IP with Network Interface Using Terraform

1. Objective

To allocate an Elastic IP address in AWS and associate it with the Network Interface created in Task 7 using Terraform.

2. Purpose

An Elastic IP (EIP) is a static public IPv4 address allocated to an AWS account.

It allows an AWS resource to be reachable through a consistent public IP address.

In this task, we allocated an Elastic IP and associated it with the Network Interface terraform-final-nic.

The Elastic IP is associated with the private IP address 10.0.1.50.

3. Prerequisites

The following resources must already exist:

Resource	Name / Configuration
VPC	terraform-final-vpc
Subnet	terraform-final-subnet
Security Group	terraform-final-sg
Network Interface	terraform-final-nic
Network Interface private IP	10.0.1.50
AWS Region	us-east-1

Terraform must already be initialized, and the existing infrastructure must be present in the Terraform state.

4. Implementation

Step 1: Open the existing Terraform project

Open the existing terraform-final-task project in VS Code. Open the **main.tf** file.

Step 2: Add the Elastic IP resource

Add the following configuration at the end of main.tf:

```
# TASK 8: Allocate and Associate Elastic IP

resource "aws_eip" "my_eip" {
  domain                   = "vpc"
  network_interface        = aws_network_interface.my_nic.id
  associate_with_private_ip = "10.0.1.50"

  tags = {
    Name = "terraform-final-eip"
  }
}
```

Step 3: Understand the configuration

- aws_eip: Terraform resource used to allocate an Elastic IP.
- domain = "vpc": Allocates the Elastic IP for use in a VPC.
- network_interface: References the Network Interface created in Task 7.
- associate_with_private_ip: Specifies the private IP address to which the Elastic IP should be associated.
- tags: Assigns a recognizable name to the Elastic IP.

Terraform uses the Network Interface reference to determine the resource dependency.

Step 4: Format the configuration: terraform fmt

Step 5: Validate the configuration: terraform validate

Step 6: Review the Terraform execution plan: terraform plan

Expected plan summary: Plan: 1 to add, 0 to change, 0 to destroy.

Step 7: Apply the configuration: terraform apply

5. Verification

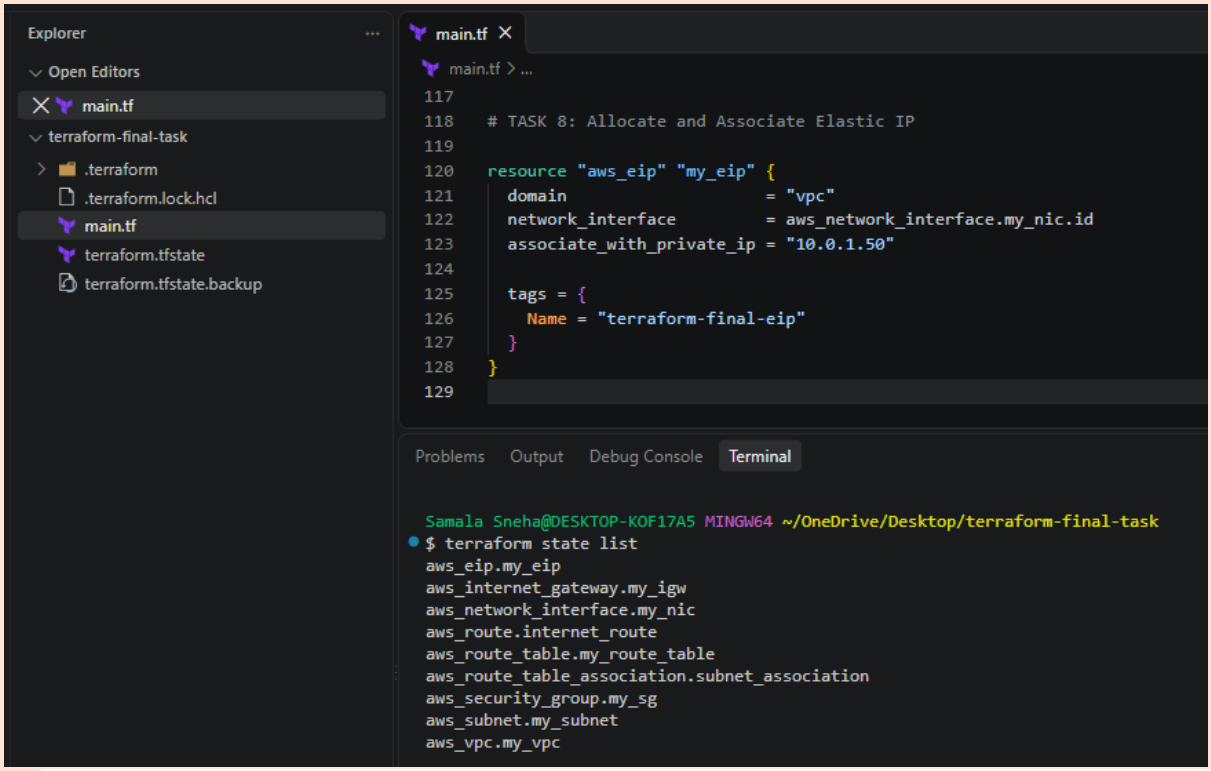
Method 1: Verify using Terraform state: terraform state list

Verified that the following resource was present: aws_eip.my_eip

Method 3: Verify using AWS Management Console

1. Open the AWS Management Console.
2. Navigate to the EC2 service.
3. Select **Elastic IPs** under Network & Security.
4. Locate the allocated Elastic IP.
5. Select the address.
6. Verify the associated Network Interface ID.
7. Verify the private IP address.

6. Result





7. Conclusion

In this task, we successfully allocated an Elastic IP using Terraform and associated it with an existing Network Interface.

The Network Interface has the private IP address 10.0.1.50, and the Elastic IP provides its public IPv4 address.

The public IP can be used to reach the Ubuntu web server after completing Task 9 and verifying the network connectivity.

Cost consideration: Elastic IPs and public IPv4 addresses may incur AWS charges. Check the current AWS pricing and release the address when it is no longer needed.

Task 9: Create Ubuntu EC2 Instance and Install Apache2 Using Terraform

1. Objective

To create an Ubuntu EC2 instance using Terraform, attach the existing Network Interface, and automatically install, configure and enable the Apache2 web server.

2. Purpose

Amazon EC2 provides scalable virtual servers in AWS.

In this task, we created an Ubuntu EC2 instance and attached the Network Interface created in Task 7.

We used Terraform user data to automate Apache2 installation during the first boot of the EC2 instance.

After installation, Apache2 was enabled to start automatically whenever the server boots.

The Elastic IP allocated in Task 8 provides a static public IPv4 address to access the web server.

4. Implementation

Step 1: Find the latest Ubuntu AMI

Added the following data source to **main.tf**:

```
data "aws_ami" "ubuntu" {
 most_recent = true
 owners      = ["099720109477"]

 filter {
  name   = "name"
  values = ["ubuntu/images/hvm-ssd-gp3/ubuntu-noble-24.04-amd64-server-*"]
 }

 filter {
  name   = "architecture"
  values = ["x86_64"]
 }

 filter {
  name   = "virtualization-type"
  values = ["hvm"]
 }

 filter {
  name   = "root-device-type"
  values = ["ebs"]
 }
}
```

This data source searches for the latest matching official Ubuntu 24.04 LTS AMI.

Step 2: Create the Ubuntu EC2 instance

Added the following resource to **main.tf**:

```
resource "aws_instance" "ubuntu_server" {
  ami           = data.aws_ami.ubuntu.id
  instance_type = "t3.micro"

  network_interface {
    network_interface_id = aws_network_interface.my_nic.id
    device_index         = 0
  }

  user_data = <<-EOF
    #!/bin/bash
    set -eux

    apt-get update -y
    DEBIAN_FRONTEND=noninteractive apt-get install -y apache2

    systemctl enable apache2
    systemctl start apache2

    echo "<h1>Sunith Application is running successfully!</h1>" > /var/www/html/index.html
  EOF

  tags = {
    Name = "terraform-final-ubuntu-server"
  }
}
```

Step 3: Understand the configuration

- aws_instance: Creates an EC2 instance.
- ami: Uses the Ubuntu AMI identified by the data source.
- instance_type: Specifies the EC2 instance size as t3.micro.
- network_interface: Attaches the existing Network Interface.
- network_interface_id: References the Network Interface created in Task 7.
- device_index = 0: Attaches the Network Interface as the primary interface.
- user_data: Executes the installation script during initial boot.
- apt-get update: Updates Ubuntu package information.
- apt-get install apache2: Installs Apache2.
- systemctl enable apache2: Enables Apache2 at system startup.
- systemctl start apache2: Starts the Apache2 service.

- index.html: Creates a custom web page.
- tags: Assigns a recognizable name to the EC2 instance.

Step 4: Format Terraform configuration: terraform fmt

Step 5: Validate Terraform configuration: terraform validate

Step 6: Review the Terraform plan: terraform plan

Expected plan summary when only Task 9 is pending: Plan: 1 to add, 0 to change, 0 to destroy.

Step 7: Create the EC2 instance: terraform apply

Step 8: Verify the EC2 instance: terraform state list

Verified that the following resource was present: aws_instance.ubuntu_server

Step 9: Verify Apache2 installation

After the EC2 instance completed its initial boot and user-data execution, accessed the website using the Elastic IP: http://3.223.149.197

Expected web page: Sunith Application is running successfully!

Step 10: Verify Apache2 service

Connected to the Ubuntu instance through SSH, when required, and executed:

sudo systemctl status apache2

Expected service status: Active: active (running)

Verified the local web server response: curl http://localhost
The server returned the custom HTML content.

5. Result

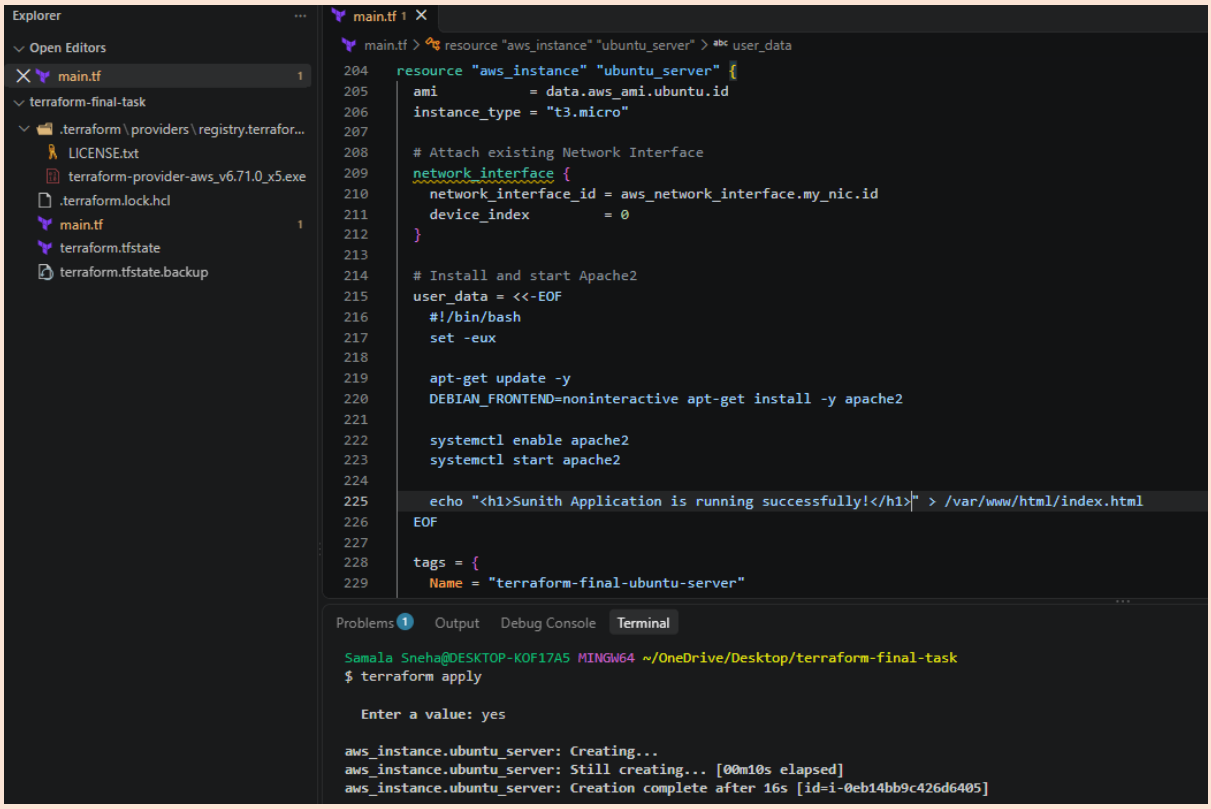
Successfully created an Ubuntu EC2 instance using Terraform.

Configured the instance with:

Setting	Value
Instance name	terraform-final-ubuntu-server
Operating system	Ubuntu 24.04 LTS
Instance type	t3.micro
Private IP	10.0.1.50
Network Interface	terraform-final-nic
Elastic IP	3.223.149.197
Web server	Apache2
Web server port	80
AWS Region	us-east-1

Apache2 was installed and configured to start automatically.

6.Implementation





7. Conclusion

In this task, we successfully created an Ubuntu EC2 instance using Terraform.

We attached the previously created Network Interface and used user data to automate Apache2 installation and configuration.

We enabled and started the Apache2 service and deployed a custom HTML page.

The Elastic IP provides a consistent public address for accessing the website.

This completes the Terraform AWS infrastructure assignment.

Complete **Main.tf**

```
# AWS PROVIDER
provider "aws" {
  region = "us-east-1"
}

# TASK 1: CREATE VPC
resource "aws_vpc" "my_vpc" {
  cidr_block         = "10.0.0.0/16"
  enable_dns_support   = true
  enable_dns_hostnames = true

  tags = {
    Name = "terraform-final-vpc"
  }
}
# TASK 2: CREATE INTERNET GATEWAY
resource "aws_internet_gateway" "my_igw" {
  vpc_id = aws_vpc.my_vpc.id

  tags = {
    Name = "terraform-final-igw"
  }
}
# TASK 3: CREATE CUSTOM ROUTE TABLE

resource "aws_route_table" "my_route_table" {
  vpc_id = aws_vpc.my_vpc.id

  tags = {
    Name = "terraform-final-rt"
  }
}
# TASK 3: CREATE INTERNET ROUTE

resource "aws_route" "internet_route" {
  route_table_id        = aws_route_table.my_route_table.id
  destination_cidr_block = "0.0.0.0/0"
  gateway_id            = aws_internet_gateway.my_igw.id
}

# TASK 4: CREATE SUBNET
resource "aws_subnet" "my_subnet" {
  vpc_id               = aws_vpc.my_vpc.id
  cidr_block            = "10.0.1.0/24"
  availability_zone      = "us-east-1a"
```

```
  map_public_ip_on_launch = true

  tags = {
    Name = "terraform-final-subnet"
  }
}

# TASK 5: ASSOCIATE SUBNET WITH ROUTE TABLE

resource "aws_route_table_association" "subnet_association" {
  subnet_id      = aws_subnet.my_subnet.id
  route_table_id = aws_route_table.my_route_table.id
}
# TASK 6: CREATE SECURITY GROUP
resource "aws_security_group" "my_sg" {
  name        = "terraform-final-sg"
  description = "Allow SSH, HTTP and HTTPS traffic"
  vpc_id      = aws_vpc.my_vpc.id

  # SSH - Only from your public IP
  ingress {
    description = "SSH from my public IP"
    from_port   = 22
    to_port     = 22
    protocol    = "tcp"
    cidr_blocks = ["124.123.159.170/32"]
  }

  # HTTP - Allow internet traffic
  ingress {
    description = "HTTP access"
    from_port   = 80
    to_port     = 80
    protocol    = "tcp"
    cidr_blocks = ["0.0.0.0/0"]
  }

  # HTTPS - Allow internet traffic
  ingress {
    description = "HTTPS access"
    from_port   = 443
    to_port     = 443
    protocol    = "tcp"
    cidr_blocks = ["0.0.0.0/0"]
  }

  # Allow outbound traffic
  egress {
```

```
    from_port   = 0
    to_port     = 0
    protocol    = "-1"
    cidr_blocks = ["0.0.0.0/0"]
  }

  tags = {
    Name = "terraform-final-sg"
  }
}

# TASK 7: CREATE NETWORK INTERFACE

resource "aws_network_interface" "my_nic" {
  subnet_id       = aws_subnet.my_subnet.id
  private_ips     = ["10.0.1.50"]
  security_groups = [aws_security_group.my_sg.id]

  tags = {
    Name = "terraform-final-nic"
  }
}

# TASK 8: ALLOCATE AND ASSOCIATE ELASTIC IP
resource "aws_eip" "my_eip" {
  domain                    = "vpc"
  network_interface         = aws_network_interface.my_nic.id
  associate_with_private_ip = "10.0.1.50"

  tags = {
    Name = "terraform-final-eip"
  }
}
# TASK 9: FIND UBUNTU 24.04 LTS AMI
data "aws_ami" "ubuntu" {
  most_recent = true
  owners      = ["099720109477"]

  filter {
    name   = "name"
    values = ["ubuntu/images/hvm-ssd-gp3/ubuntu-noble-24.04-amd64-server-*"]
  }

  filter {
    name   = "architecture"
    values = ["x86_64"]
  }
```

```
  filter {
    name   = "virtualization-type"
    values = ["hvm"]
  }

  filter {
    name   = "root-device-type"
    values = ["ebs"]
  }
}
# TASK 9: CREATE UBUNTU EC2 AND INSTALL APACHE2
resource "aws_instance" "ubuntu_server" {
  ami           = data.aws_ami.ubuntu.id
  instance_type = "t3.micro"

  # Attach existing Network Interface
  network_interface {
    network_interface_id = aws_network_interface.my_nic.id
    device_index         = 0
  }

  # Install and start Apache2
  user_data = <<-EOF
    #!/bin/bash
    set -eux

    apt-get update -y
    DEBIAN_FRONTEND=noninteractive apt-get install -y apache2

    systemctl enable apache2
    systemctl start apache2

    echo "<h1>Sunith Application is running successfully!</h1>" > /var/www/html/index.html
  EOF

  tags = {
    Name = "terraform-final-ubuntu-server"
  }
}
```